Production-Grade Retrieval-Augmented Generation (RAG) System

Technical Documentation & Architecture Guide

Building Trustworthy AI Systems with Evidence-Based Answers

Document Version	1.0.0
Date	June 19, 2026
Author	AI Engineering Team
Classification	Technical Reference

This document provides a comprehensive overview of the Production-Grade RAG System, including architecture, components, data flow, and implementation strategy.

Table of Contents

1. Executive Summary

The Production-Grade Retrieval-Augmented Generation (RAG) System is a comprehensive enterprise-grade solution designed to deliver accurate, evidence-based answers from an organization's private knowledge base. Unlike traditional chatbots that rely solely on a language model's parametric knowledge, this system retrieves relevant information from ingested documents and generates responses grounded in verifiable sources.

The system addresses a critical challenge in enterprise AI adoption: hallucination. By combining dense vector retrieval with sparse keyword search (BM25), re-ranking with cross-encoders, and a sophisticated LangGraph workflow, the system ensures that every answer is traceable to specific source documents. Each response includes inline citations, allowing users to verify the accuracy of the information.

> **Core Philosophy: Every answer must be backed by evidence. No hallucination. No guessing. Every claim traceable to a source document with a verifiable citation.**

1.1 Key Capabilities

- **Document Ingestion Pipeline —** Process PDFs, HTML pages, Markdown files, and web content with automated chunking and metadata extraction.

- **Hybrid Search (BM25 + Dense Vectors) —** Combines lexical keyword matching with semantic similarity using Reciprocal Rank Fusion (RRF) for optimal retrieval.
- **Cross-Encoder Re-Ranking —** A second-stage re-ranker (BAAI/bge-reranker-v2-m3) scores retrieved chunks for maximum precision.
- **LangGraph Workflow —** State machine orchestrates query transformation, retrieval, re-ranking, generation, hallucination checking, and citation formatting.
- **Citation System —** Every answer segment is linked to its source document with page numbers and direct references.
- **Evaluation Suite —** Automated metrics (Recall@k, MRR, Faithfulness, Answer Relevance) with RAGAS framework integration.
- **Production Infrastructure —** Docker Compose orchestration, JWT authentication, Langfuse monitoring, structured logging, and CI/CD pipelines.

2. Project Overview

2.1 What is Retrieval-Augmented Generation?

Retrieval-Augmented Generation (RAG) is an AI architecture that enhances large language models with external knowledge retrieval. Instead of generating answers from the model's training data alone, RAG first searches a knowledge base for relevant information, then provides this context to the LLM for answer generation. This approach significantly reduces hallucination, enables answers from private or proprietary data, and allows for real-time knowledge updates without model retraining.

2.2 System Design Philosophy

This system is built on five core design principles:

1. Accuracy First — Every component is optimized for retrieval precision and answer faithfulness.
2. Traceability — All answers must include verifiable citations to source documents.
3. Scalability — Containerized microservices architecture that scales horizontally.
4. Observability — Full monitoring, tracing, and logging with Langfuse integration.
5. Cost Efficiency — Local embedding model (BGE-base-en-v1.5) and local LLM (Qwen3 8B) eliminate API costs.

2.3 Target Audience

- **Organizations —** Companies needing to make their internal documentation searchable and answerable.
- **Developers —** Teams building knowledge-base assistants, customer support bots, or document Q&A systems.
- **Researchers —** Academics and analysts needing to query large collections of research papers.
- **Enterprises —** Legal, healthcare, and finance sectors requiring citation-backed answers.

3. System Architecture

The system follows a layered microservices architecture with clear separation of concerns. Each component is containerized and independently scalable. The architecture is designed for production deployments with authentication, monitoring, and CI/CD integration.

3.1 High-Level Architecture

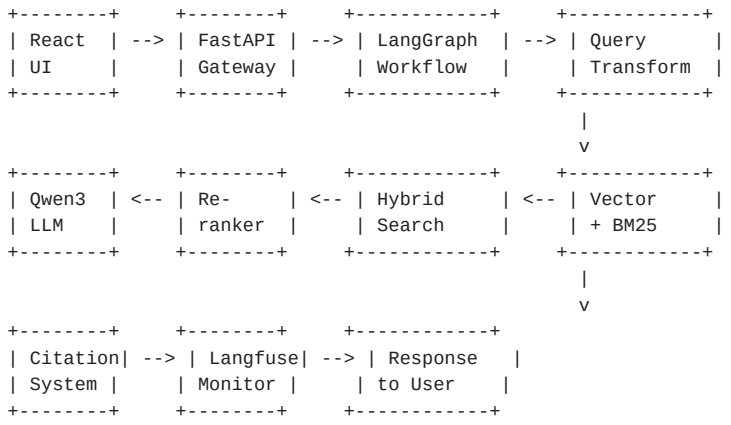
Layer	Component	Technology	Responsibility
Presentation	React Frontend	React + TypeScript + TailwindCSS	Chat UI, document upload, citation display
API Gateway	FastAPI Server	FastAPI + OAuth2 + Structlog	Authentication, routing, request validation
Orchestration	LangGraph Workflow	LangChain + LangGraph	Query processing, retrieval, generation flow
Retrieval	Hybrid Search	Qdrant + BM25 (rank-bm25)	Dense vector + sparse keyword search
Retrieval	Re-ranker	BAAI/bge-reranker-v2-m3	Cross-encoder precision scoring
Embeddings	Embedding Service	BGE-base-en-v1.5 + sentence-transformers	Document and query vectorization
Generation	LLM Engine	Qwen3 8B via Ollama	Context-grounded answer generation
Monitoring	Observability	Langfuse + Prometheus + Grafana	Tracing, metrics, cost tracking
Infrastructure	Container Orchestration	Docker Compose	Service management, networking, volumes

3.2 Technology Stack Summary

Component	Technology Choice
Backend Language	Python 3.12+
Web Framework	FastAPI (async)
RAG Framework	LangChain + LangGraph
Vector Database	Qdrant (self-hosted)
Embedding Model	BAAI/bge-base-en-v1.5 (768 dimensions)
Large Language Model	Qwen3 8B (via Ollama)
Re-ranker Model	BAAI/bge-reranker-v2-m3
Hybrid Search	BM25 + Dense Vector (RRF fusion)
Frontend	React 18 + TypeScript + TailwindCSS
Monitoring	Langfuse (LLM observability)
Evaluation	RAGAS (automated metrics)
Authentication	JWT-based OAuth2
Testing	pytest + pytest-cov
CI/CD	GitHub Actions
Containerization	Docker + Docker Compose
Logging	Structlog (structured logging)

3.3 Architecture Diagram

The following diagram illustrates the request flow from user query to generated response:



4. Core Components

4.1 Document Ingestion Pipeline

The ingestion pipeline is responsible for loading documents from various sources, extracting text content, splitting them into semantically meaningful chunks, generating embeddings, and storing them in Qdrant. The pipeline supports multiple document formats and provides metadata extraction for enhanced retrieval.

Supported Document Formats

- **PDF —** Using PyMuPDF (fitz) for text extraction with layout preservation.
- **HTML —** BeautifulSoup-based parser for web pages with tag-aware extraction.
- **Markdown —** Native markdown support with frontmatter metadata parsing.
- **Plain Text —** Direct text file ingestion with encoding detection.

Chunking Strategy

Documents are split using a two-stage approach. First, RecursiveCharacterTextSplitter divides the text at sensible boundaries (paragraphs, sentences, then characters). Second, a semantic splitter ensures chunks maintain topical coherence. Default configuration: chunk_size=512 tokens, chunk_overlap=50 tokens.

Metadata Extraction

Each chunk is enriched with metadata including: document title, source URL or file path, page number, section heading hierarchy, chunk index, creation timestamp, and document type. This metadata enables fine-grained filtering during retrieval and accurate citation generation.

4.2 Embedding Pipeline

The embedding service uses the BAAI/bge-base-en-v1.5 model from the BAAI family. This model produces 768-dimensional dense vectors optimized for retrieval tasks. The model is selected for its strong performance on

the MTEB (Massive Text Embedding Benchmark) and its efficient inference speed.

Model Specifications

Model	BAAI/bge-base-en-v1.5
Dimensions	768
Max Sequence Length	512 tokens
Normalization	L2-normalized (suitable for cosine similarity)
Framework	sentence-transformers
Performance	MTEB Average: 63.7
Inference	CPU-optimized, ~50 docs/sec on modern hardware

Caching Strategy

Embeddings are cached on disk using a key-value store (diskcache). When a document chunk is re-ingested, its hash is computed and checked against the cache. This avoids redundant computation for unchanged documents and significantly speeds up re-indexing operations.

4.3 Vector Database — Qdrant

Qdrant serves as the primary vector store. It is an open-source, high-performance vector similarity search engine written in Rust. Qdrant is chosen for its production-grade features: filtering, payload storage, HNSW index configuration, and built-in quantization for memory efficiency.

Collection Configuration

- **Vector Size —** 768 dimensions (matching BGE-base-en-v1.5)
- **Distance Metric —** Cosine similarity
- **Index —** HNSW (Hierarchical Navigable Small World) with m=16 and ef_construct=200
- **Quantization —** Scalar quantization for 4x memory reduction
- **Payload —** Full metadata stored for filtering and citation
- **Write-Ahead Log —** Enabled for crash recovery and durability

Query Process

On query, the embedding model encodes the user question into a 768-dimensional vector. Qdrant performs a nearest neighbor search using cosine similarity with the HNSW index. The search returns the top-k most similar chunks along with their payload data. The system retrieves k=20 chunks initially, which are then filtered and re-ranked in subsequent stages.

4.4 Hybrid Search (BM25 + Dense Retrieval)

Hybrid search combines the strengths of keyword-based (lexical) search and semantic (dense) search. BM25 excels at exact keyword matching and handling rare terms, while dense retrieval captures semantic similarity even when there is no lexical overlap. The fusion of both approaches significantly improves retrieval robustness.

Reciprocal Rank Fusion (RRF)

The results from BM25 and dense vector search are merged using the RRF algorithm. Each document receives a combined score calculated as the sum of reciprocal ranks from both result lists. The formula is: RRF(d) = 1/(k + rank_bm25(d)) + 1/(k + rank_dense(d)), where k=60 is a constant. This method produces stable, high-quality merged rankings without score normalization.

Query Transformation

Before retrieval, the user's query undergoes transformations to improve search quality. Three techniques are applied: (1) query expansion — generating alternative phrasings using the LLM, (2) HyDE (Hypothetical Document Embeddings) — generating a hypothetical ideal document and embedding it for retrieval, and (3) multi-query search — executing multiple query variants and merging results.

4.5 Re-Ranking System

After hybrid search retrieves the top-k candidates, a cross-encoder re-ranker scores each chunk against the original query. Unlike bi-encoders (used in dense retrieval), cross-encoders process the query and chunk together through a transformer, producing a more accurate relevance score.

Re-Ranker Specifications

- **Model —** BAAI/bge-reranker-v2-m3
- **Architecture —** Cross-encoder (query + document processed together)
- **Input —** Top 20 chunks from hybrid search
- **Output —** Relevance scores 0-1, re-ranked list
- **Top-k Selection —** Top 3-5 most relevant chunks passed to generation

4.6 LangGraph Workflow Engine

LangGraph is the orchestration layer that manages the entire query-to-answer pipeline as a state machine. Unlike linear chains, LangGraph supports branching, loops, conditional logic, and parallel execution. This enables sophisticated behaviors like self-correction, hallucination checking, and fallback handling.

Workflow Nodes

Node Name	Description
query_parse	Parse and validate user input, extract metadata filters
query_transform	Generate query variants and hypothetical document
hybrid_search	Execute BM25 + dense vector search in parallel
re_rank	Score and re-rank retrieved chunks
hallucination_check	Verify retrieved context is sufficient for answer
generate	Generate answer with citations using LLM
format_response	Structure response with citations and metadata
log_trace	Send trace data to Langfuse

State Machine Design

The workflow uses a StateGraph with typed state schema. Each node reads from and writes to a shared state dictionary. Conditional edges enable branching: if hallucination_check fails, the workflow routes to a fallback

handler instead of generation. If multiple query variants are used, hybrid_search runs in parallel branches.

4.7 Generation Engine — Qwen3 8B

The generation component uses Qwen3 8B, a state-of-the-art open-source language model from Alibaba Cloud. The model runs locally via Ollama, eliminating API costs and ensuring data privacy. Qwen3 8B provides strong instruction following, long context handling (up to 128K tokens), and multilingual capabilities.

Prompt Strategy

The generation prompt follows a structured template: system instruction specifying the role as a documentation assistant, instructions to answer only from provided context, citation format requirements, and a "don't know" fallback. The retrieved chunks are injected as context, and the user query is appended. Temperature is set to 0.1 for deterministic, factual responses.

Response Format

The LLM generates a structured response with inline citations. Each factual claim is followed by a citation number in square brackets, referencing the source document. The generation is streamed token by token via Server-Sent Events (SSE) from FastAPI to the frontend for real-time display.

4.8 Citation System

The citation system is a critical component that ensures answer traceability. It maps each generated claim to its source document chunk, providing users with verifiable evidence.

Citation Format

Citations follow the format [N] where N is a number referencing a source document. At the end of each answer, a "Sources" section lists all cited documents with titles, page numbers, and direct links. Example: [1] Next.js Documentation - dynamic-routes.md - Page 12

Chunk-to-Source Mapping

When the LLM generates a citation token, the system maps it to the corresponding chunk metadata. This ensures that every citation can be traced back to a specific document, page, and section. The frontend renders citations as clickable links that open the source document.

5. Data Flow

5.1 Ingestion Flow

The document ingestion process follows these sequential steps:

1. Document Upload — User uploads a file via the React frontend or API endpoint. Supported formats: PDF, HTML, MD, TXT.
2. Content Extraction — The document loader extracts raw text, preserving structure where possible. PDFs use PyMuPDF, HTML uses BeautifulSoup.

3. Text Chunking — The extracted text is split into overlapping chunks using the two-stage strategy (recursive + semantic).
4. Embedding Generation — Each chunk is passed through BGE-base-en-v1.5 to produce a 768-dimensional vector.
5. Metadata Enrichment — Chunk metadata is assembled: title, source, page number, section hierarchy, etc.
6. Qdrant Storage — Chunks and their vectors are upserted into the Qdrant collection with full payload.
7. Indexing Confirmation — Qdrant confirms storage, and the user receives a success notification.

5.2 Query Flow

When a user submits a query, the following pipeline executes:

1. User Query — User types a question in the React chat interface (e.g., "How do I implement dynamic routing in Next.js?").
2. API Reception — FastAPI receives the query, authenticates the user, and passes it to the LangGraph workflow.
3. Query Transformation — The query is expanded into variants and optionally used to generate a hypothetical document (HyDE).
4. Hybrid Search — Dense vector search (Qdrant) and BM25 keyword search run in parallel. Both return top-20 results.
5. RRF Fusion — Reciprocal Rank Fusion merges the two result sets into a single ranked list.
6. Re-Ranking — The cross-encoder re-ranker scores the top-20 chunks against the original query, selecting top-3 to top-5.
7. Hallucination Check — The system verifies that retrieved chunks contain sufficient relevant information to answer the query.
8. LLM Generation — Qwen3 8B receives the query + context and generates an answer with inline citations.
9. Citation Formatting — The output processor formats citations and appends the source reference section.
10. Streaming Response — The response is streamed token-by-token via SSE to the frontend.
11. Monitoring Log — Langfuse records the entire trace: query, retrieval scores, LLM call, latency, tokens used.
12. User Display — The React UI renders the answer with clickable citations, source cards, and relevance scores.

6. API Layer

The FastAPI backend exposes a RESTful API for all system operations. The API is designed with async endpoints for non-blocking I/O, automatic OpenAPI documentation, request validation via Pydantic, and JWT-based authentication.

6.1 API Endpoints

Method	Endpoint	Description

Method	Endpoint	Description
POST	/api/auth/register	Register new user account
POST	/api/ingest/upload	Upload document(s) for ingestion
GET	/api/ingest/status/{id}	Check ingestion status
POST	/api/query/chat	Submit a query, get streamed response
POST	/api/query/search	Submit a query, get raw search results
GET	/api/documents	List all ingested documents
GET	/api/documents/{id}	Get document details and chunks
DELETE	/api/documents/{id}	Delete document and its chunks
GET	/api/evaluate/run	Run evaluation suite on test dataset
GET	/api/evaluate/results	Get evaluation results and metrics
GET	/api/health	System health check
GET	/api/stats	System usage statistics

6.2 Authentication & Authorization

Access to the API is secured using JSON Web Tokens (JWT). Users authenticate via the /auth/login endpoint and receive an access token (15-minute expiry) and a refresh token (7-day expiry). Protected endpoints validate the token via OAuth2 dependency injection. Role-based access control (RBAC) distinguishes admin users (can ingest/delete) from regular users (can query only).

7. Frontend Application

The frontend is a modern single-page application built with React 18 and TypeScript. It provides an intuitive chat interface with real-time streaming responses, interactive citations, and document management capabilities. The UI is styled with TailwindCSS for a clean, responsive design.

7.1 Key Features

- **Chat Interface —** Message-based Q&A with streaming token-by-token response display. Markdown rendering for formatted answers.
- **Citation Cards —** Each answer includes collapsible citation cards showing source document title, page number, and a preview of the relevant chunk.
- **Document Upload —** Drag-and-drop interface for uploading documents. Real-time progress tracking and ingestion status updates.
- **Dashboard —** System statistics: total documents, queries today, average latency, token usage, and retrieval success rate.
- **Document Manager —** Browse, search, and delete ingested documents. View document details and chunk previews.
- **Authentication UI —** Login and registration pages with form validation and session management.

- **Dark/Light Mode —** Theme toggle for comfortable viewing in any environment.

7.2 Component Architecture

Component	Description
App	Root component, routing, theme provider
ChatPage	Main chat interface with message list and input
MessageList	Virtualized message list with streaming support
MessageBubble	Individual message with markdown and citations
CitationCard	Collapsible source document card
UploadPage	File upload with drag-drop and progress
Dashboard	System statistics and charts
DocManager	Document list with search and delete
LoginForm	User authentication form
Sidebar	Navigation sidebar
Layout	Page layout with header, sidebar, content

8. Evaluation Framework

A rigorous evaluation framework ensures the system meets quality standards before deployment. The framework includes a golden test dataset, automated metric computation using the RAGAS library, and regression testing integrated into the CI pipeline.

8.1 Golden Dataset

The evaluation dataset consists of 50+ curated question-answer pairs derived from the ingested documentation. Each entry includes: question, ground truth answer, relevant document IDs, relevant chunk IDs, and expected citation sources. The dataset covers simple factual questions, comparative questions, multi-document questions, and edge cases.

8.2 Retrieval Metrics

Metric	Description	Target
Recall@k	Proportion of relevant documents retrieved in top-k	Target > 0.85 @ k=5
MRR	Mean Reciprocal Rank — rank position of first relevant result	Target > 0.90
NDCG@k	Normalized Discounted Cumulative Gain	Target > 0.80 @ k=5
Precision@k	Proportion of retrieved documents that are relevant	Target > 0.70 @ k=5

8.3 Generation Metrics

Metric	Description	Target
Faithfulness	Proportion of claims in answer that are supported by context	Target > 0.90
Answer Relevance	How relevant the answer is to the question	Target > 0.85

Metric	Description	Target
Context Recall	Proportion of relevant context that was retrieved	Target > 0.85
BLEU Score	N-gram overlap between generated and reference answer	Target > 0.30
ROUGE-L	Longest common subsequence between generated and reference	Target > 0.45

8.4 Automated Evaluation Pipeline

The evaluation pipeline runs automatically on every pull request via GitHub Actions. When triggered, the pipeline: (1) builds the system, (2) ingests the golden dataset documents, (3) runs all test queries, (4) computes retrieval and generation metrics, (5) compares against baseline thresholds, and (6) reports pass/fail status. Any regression in metrics blocks the PR from merging.

9. Production Deployment

9.1 Docker Compose Architecture

The entire system is containerized using Docker and orchestrated with Docker Compose. Each service runs in its own container with explicit resource limits, health checks, and dependency management. The compose file defines the following services:

Service	Description	Port
frontend	React app served via Nginx	Port 3000
backend	FastAPI application (uvicorn)	Port 8000
qdrant	Qdrant vector database	Port 6333
ollama	Ollama LLM server	Port 11434
langfuse	Langfuse monitoring server	Port 3001
postgres	Langfuse database (PostgreSQL)	Port 5432
nginx	Reverse proxy (API gateway)	Port 80/443

9.2 CI/CD Pipeline — GitHub Actions

Two GitHub Actions workflows ensure code quality and automated deployment:

CI Pipeline (ci.yml)

- **Trigger —** On every push and pull request to main branch.
- **Steps —** Checkout -> Set up Python -> Install dependencies -> Run ruff (linting) -> Run mypy (type checking) -> Run pytest (unit tests + integration tests) -> Build Docker images.
- **Cache —** Python venv, Docker layers, and model downloads are cached for fast execution.

CD Pipeline (cd.yml)

- **Trigger —** On merge to main branch (after CI passes).

- **Steps —** Build and tag Docker images -> Push to container registry -> Deploy to staging -> Run smoke tests -> Promote to production.
- **Rollback —** Automatic rollback if smoke tests fail or health checks return non-200 status.

9.3 Monitoring & Observability — Langfuse

Langfuse provides comprehensive observability for the LLM pipeline. It captures:

- **Traces —** End-to-end request tracing from query to response, with timing for each step.
- **LLM Calls —** Prompt and completion logging, token counts, and cost estimation.
- **Retrieval Metrics —** Number of chunks retrieved, scores, re-ranking impact.
- **Evaluation Scores —** Inference-time evaluation of answer faithfulness and relevance.
- **Latency Analysis —** P50, P95, and P99 latency for each pipeline stage.
- **User Feedback —** Thumbs up/down for each answer to collect training data for improvement.

10. Use Cases

10.1 1. SaaS Documentation Assistant

The primary use case is answering questions about software documentation. By ingesting documentation from frameworks like React, Next.js, LangChain, and FastAPI, developers can get instant, accurate answers with source references.

- **Example Query —** "How do I implement dynamic routing in Next.js?"
- **System Action —** Retrieves the relevant section from Next.js docs about dynamic routes, generates an answer with code examples, and cites the exact documentation page.
- **Value —** Eliminates hours of manual documentation searching. Developers get answers in seconds with confidence in accuracy.

10.2 2. Research Paper Assistant

Researchers can upload collections of academic papers and query the system for key findings, methodologies, and comparisons.

- **Example Query —** "What are the main contributions of paper #7 regarding attention mechanisms?"
- **Value —** Enables rapid literature review across hundreds of papers with verifiable citations to specific paragraphs.

10.3 3. Legal Document Assistant

Law firms and legal departments can ingest contracts, agreements, and legal documents for instant clause retrieval.

- **Example Query —** "What is the termination notice period in the service agreement with vendor

X?"

- **Value —** Reduces contract review time from hours to seconds with exact clause citations.

10.4 4. Healthcare Policy Assistant

Healthcare organizations can query internal policies, treatment guidelines, and regulatory documents.

- **Example Query —** "What is the recommended protocol for patient data access under HIPAA?"
- **Value —** Ensures compliance by grounding answers in official policy documents with exact references.

10.5 5. University Student Assistant

Universities can ingest handbooks, course catalogs, and policy documents for student self-service.

- **Example Query —** "How many credit hours are required for the Computer Science major?"
- **Value —** Reduces administrative workload by enabling students to find answers independently.

11. Implementation Roadmap

The project is organized into seven implementation phases, each building on the previous one. Estimated total time: 17-23 days of part-time work.

Phase	Name	Key Activities	Duration
1	Foundation	Project setup, Docker Compose, config, loaders, splitters, embedding pipeline, Qdrant integration	3-4 days
2	Retrieval Stack	BM25 implementation, RRF fusion, cross-encoder re-ranker, query transformation	2-3 days
3	LangGraph & Generation	StateGraph workflow, prompt engineering, citation system, streaming	3-4 days
4	API & Authentication	FastAPI routes, JWT auth, request validation, rate limiting	2 days
5	Frontend Application	React UI, chat interface, citation cards, document upload, dashboard	3-4 days
6	Evaluation & Testing	Golden dataset, RAGAS metrics, pytest suite, regression testing	2-3 days
7	Production Readiness	CI/CD pipelines, Docker optimization, monitoring config, documentation	2-3 days

12. Success Metrics & Acceptance Criteria

The following metrics define project success and serve as acceptance criteria for production deployment:

Metric	Target	Description
Retrieval Recall@5	> 0.85	Proportion of relevant documents in top-5 results
Answer Faithfulness	> 0.90	Claims supported by retrieved context

Metric	Target	Description
P50 Latency	< 3 seconds	Median query-to-response time
P95 Latency	< 8 seconds	95th percentile response time
Citation Accuracy	> 95%	Citations correctly map to source documents
System Uptime	> 99.5%	API and frontend availability
Test Coverage	> 80%	Code coverage from pytest suite
CI Pass Rate	100%	All CI checks pass before merge
Token Efficiency	< 1000 tokens/response	Average tokens per generated answer

12.1 Monitoring Dashboard Metrics

The Grafana dashboard displays real-time metrics including: queries per minute, average latency by stage, retrieval score distribution, token consumption, error rates by endpoint, and active users. Alerts are configured for: latency exceeding P95 threshold, error rate > 5%, and service health check failures.

13. Conclusion

The Production-Grade RAG System represents a complete, enterprise-ready solution for building trustworthy AI-powered knowledge assistants. By combining state-of-the-art retrieval techniques with rigorous evaluation and production infrastructure, the system delivers accurate, verifiable answers from organizational knowledge bases.

Key differentiators of this system include: hybrid search with RRF fusion for robust retrieval, cross-encoder re-ranking for precision, LangGraph orchestration for sophisticated workflow control, a comprehensive citation system for answer traceability, local LLM inference for cost efficiency and data privacy, automated evaluation with RAGAS metrics, and full production infrastructure with monitoring, logging, and CI/CD.

> **Portfolio Impact: This project demonstrates capability across the full AI engineering stack: data processing, vector databases, LLM orchestration, evaluation, monitoring, and production deployment. It showcases not just "chatbot building" but production-grade AI system engineering.**

This document is intended as a technical reference and project overview. For implementation details, refer to the project source code and inline documentation.